

CSCI 361 Lecture 12:

Decidable Languages

Shikha Singh

Announcements & Logistics

- HW 6 is due tomorrow at 10 pm
- Hand in **Exercise 9**, and pick up **Exercise 10**
- **Midterm 2**: Thursday, Nov 6
 - Topics in Chapter 2 onwards (HW 4-7) will be the focus
 - Similar format as Midterm 1

Last Time

- Equivalence of different TM models:
 - Multi-tape, NTM, etc
- Church-Turing thesis:
 - Any algorithm can be implemented on a Turing machine

Today

- Example of decidable languages
- Approaching undecidability
 - How to prove some languages are not solvable by any TM (and thus no algorithm exists for them by CT thesis)

Turing Point in Course: Inputs

- So far, our inputs were simple, e.g. binary strings
- Now our input can be an encoding of any object
 - E.g., if A is a graph, $\langle A \rangle$ represents a valid encoding of the graph such as vertices followed by a listing of all edges
 - If M is a DFA, then $\langle M \rangle$ is an encoding of the DFA which lists all 5 tuples, including an enumeration of the transitions $\delta : Q \times \Sigma \rightarrow Q$
- When a TM receives such an input, we assuming its encoding is written on the TM tape in some reasonable format

Turing Point in Course: Algorithms

- We use the CT thesis to assume that our TM can execute any well-defined algorithms or well-specified programs
- For example, give an NFA description, it can convert an NFA to a DFA using the power set construction algorithm

Decidability and Undecidability

- To prove a language L is decidable we must give a TM M such that
 - $w \in L \iff M$ halts and accepts w
- Often will reduce to other decidable problems
- To prove a language L is undecidable we must show that no TM exists that can decide L
 - Will discuss undecidable problems next lecture

Example Problems of Interest

- In general, is it useful to be able to check a program's output on a given input: constantly used in verification
- Acceptance problems: is this string accepted by this machine?
 - $A_{\text{DFA}} = \{\langle M, w \rangle \mid M \text{ is a DFA and } w \in L(M)\}$
 - Similar variant for NFA and regular expression
 - $A_{\text{CFG}} = \{\langle G, w \rangle \mid G \text{ is a CFG and } w \in L(G)\}$
 - Similar variant for PDA
 - **(Next class).** $A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM and } w \in L(M)\}$

Acceptance Problem for DFAs

Theorem. The language

$A_{\text{DFA}} = \{ \langle M, w \rangle \mid M \text{ is a DFA and } w \in L(M) \}$ is decidable.

- Assume the DFA $M = (Q, \Sigma, \delta, q_0, F)$ is encoded as a string consisting of the states, followed by alphabet, followed by a list of triples $(q, x, \delta(q, x))$, start state and final states (each separated by #)
- Simulate the DFA M on w (how would this work?)
- Accept if M accepts, otherwise reject

Using Algorithms and Subroutines

Theorem. The following languages are decidable.

$$A_{\text{NFA}} = \{ \langle N, w \rangle \mid N \text{ is a NFA and } w \in L(N) \}$$

$$A_{\text{REX}} = \{ \langle R, w \rangle \mid R \text{ is a regular expression and } w \in L(R) \}$$

What about CFGs?

Theorem. $A_{CFG} = \{\langle G, w \rangle \mid G \text{ is a CFG and } w \in L(G)\}$ is decidable.

Question. How do we check if a grammar accepts a given string?

- Trying all possible derivations does not work since some derivations may not terminate (need a decider)
- E.g., if the grammar has unnecessary rules that don't reach a base case and correspond to infinite recursion, e.g. $T \rightarrow T$

What about CFGs?

Theorem. $A_{CFG} = \{\langle G, w \rangle \mid G \text{ is a CFG and } w \in L(G)\}$ is decidable.

Chomsky Normal Form. Each grammar G can be converted into a canonical "nice" form called Chomsky Normal Form where each rule is of the form $A \rightarrow BC$ or $A \rightarrow a$ where A, B, C are variables and a is a terminal. Additional if $\varepsilon \in L(G)$ then $S \rightarrow \varepsilon$ is allowed.

Nice property we can use: Every derivation of string $w \in L(F)$ of length n takes exactly $2n - 1$ steps

What about CFGs?

Theorem. $A_{CFG} = \{\langle G, w \rangle \mid G \text{ is a CFG and } w \in L(G)\}$ is decidable.

Algorithm we can use without going into it: there is an algorithm to convert any grammar G into a grammar G' that is in CNF such that $L(G) = L(G')$.

What about CFGs?

Theorem. $A_{CFG} = \{\langle G, w \rangle \mid G \text{ is a CFG and } w \in L(G)\}$ is decidable.

Proof.

On input $\langle G, w \rangle$ where G is a CFG:

- Convert G into G' which is in CNF
- Try all possible derivations of length $2n - 1$ using rules of G'
- If w is generated by any of these, *accept*. Otherwise, *reject*.

What about TMs?

Question. Is $A_{\text{TM}} = \{ \langle M, w \rangle \mid T \text{ is a TM and } w \in L(M) \}$ decidable?



Empty-ness Checks

- Does this program to solve a decision problem ever output 1?
- Emptiness problems:
 - $E_{\text{DFA}} = \{\langle M \rangle \mid M \text{ is a DFA and } L(M) = \emptyset\}$
 - Similar variant for NFA and regular expression
 - $E_{\text{CFG}} = \{\langle G \rangle \mid G \text{ is a CFG and } L(G) = \emptyset\}$
 - Similar variant for PDA
 - **(Next class).** $E_{\text{TM}} = \{\langle M, w \rangle \mid T \text{ is a TM and } L(M) = \emptyset\}$

Emptiness Testing

Theorem. $E_{\text{DFA}} = \{\langle M \rangle \mid M \text{ is a DFA and } L(M) = \emptyset\}$ is decidable.

Question. How do we check if a DFA never accepts a string?

Emptiness Testing for DFAs

Theorem. $E_{\text{DFA}} = \{\langle M \rangle \mid M \text{ is a DFA and } L(M) = \emptyset\}$ is decidable.

Proof. TM that decides this language:

On input $\langle M \rangle$ where M is a DFA:

- Mark the start state of M
- Repeat until no new states are marked:
 - Mark any state that has a transition coming from a marked state
- If no accept state is marked, accept; otherwise reject

Emptiness Testing for CFGs

Theorem. $E_{\text{CFG}} = \{\langle G \mid G \text{ is a CFG and } L(G) = \emptyset \}$ is decidable.

Question. How do we check if a grammar never generates a string?

Emptiness Testing for CFGs

Theorem. $E_{\text{CFG}} = \{\langle G \mid G \text{ is a CFG and } L(G) = \emptyset\}$ is decidable.

Proof. TM that decides this language:

On input $\langle G \rangle$ where G is a CFG:

- Mark all the terminals in G
- Repeat until no new variables get marked:
 - Mark any variable $A \rightarrow u_1 u_2 \cdots u_k$ such that each u_i on the RHS is already marked
- If start variable S is marked, accept; otherwise reject

Class Exercise

Problem. Show that

$ALL_{DFA} = \{ \langle M \rangle \mid M \text{ is a DFA and } L(M) = \Sigma^* \}$ is decidable.

$EQ_{DFA} = \{ \langle M, N \rangle \mid M, N \text{ are DFAs and } L(M) = L(N) \}$ is decidable.

Hint. Try to reduce it to one of the problems we have already discussed!

What about CFG Equivalence?

Question. Is $EQ_{CFG} =$

$\{\langle G_1, G_2 \rangle \mid G_1, G_2 \text{ are CFGs and } L(G_1) = L(G_2)\}$ decidable?



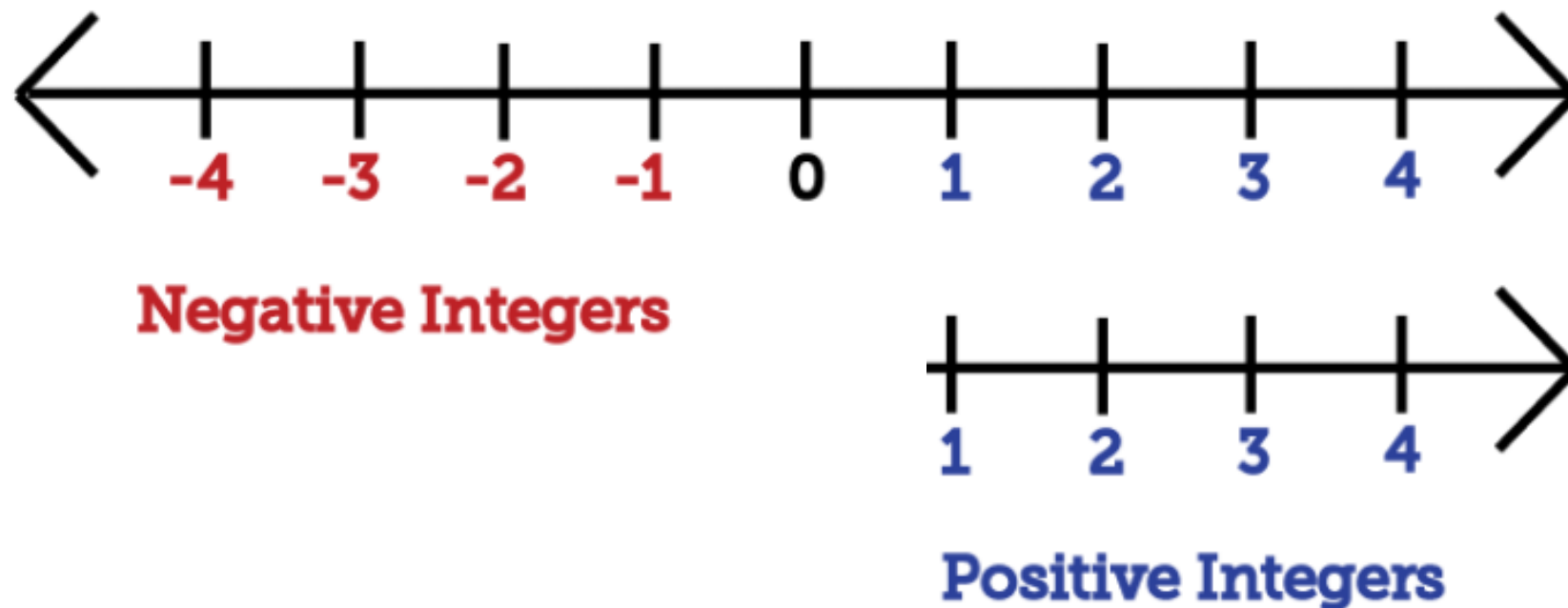
Undecidability Detour: Countability

Review: Cardinality

- Sets can be finite or infinite
- S is **finite** if there is a **bijection** function $f: S \rightarrow \{1, 2, \dots, n\}$ for some $n \in \mathbb{N}$
- Denote the size or cardinality as $|S| = n$
- S is **infinite** if no such bijection exists
- S is a **countably infinite set** if there is a **bijection** function $f: S \rightarrow \mathbb{N}$
- S is a **countable** if it is finite or countably infinite
- S is a **uncountable** if it is not countable

Proof by Construction Example

- **Theorem:** The set of integers $\mathbb{Z}$ is countable.
- Proof Outline:
 - Need to construct a bijective function f from $\mathbb{Z}$ to $\mathbb{N}$
 - Show that it is one-one and onto
 - Any ideas?



Proof by Construction Example

- **Theorem:** The set of integers $\mathbb{Z}$ is countable.
- Proof Outline:
 - Need to construct a bijective function f from $\mathbb{Z}$ to $\mathbb{N}$
 - Show that it is one-one and onto

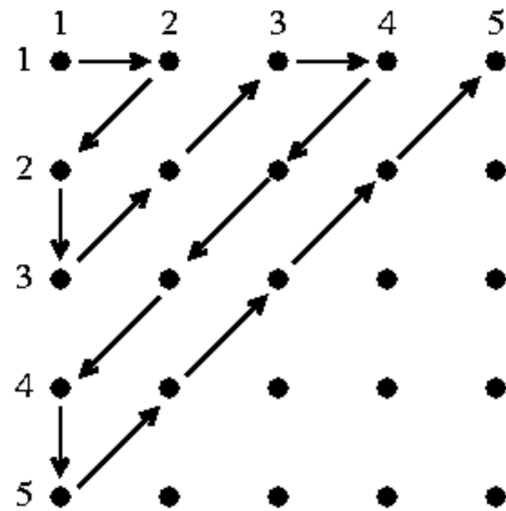
$$f(x) = \begin{cases} -2x & \text{if } x < 0 \\ 2x + 1 & \text{if } x \geq 0 \end{cases}$$

Countability

- Any set that can be "enumerated" using natural numbers is countable
 - Can talk about 1st item, 2nd item and so on
 - All items of the set appear in this on this list
- The set of all strings over a finite alphabet Σ is countable
 - Recall $\Sigma^* = \bigcup_{n \in \mathbb{N} \cup \{0\}} \Sigma^n$
 - Here Σ^n is a set of all strings of length exactly n over Σ
 - Each Σ^n is finite and contains $\ell = |\Sigma|^n$ elements, can list them in lexicographic order at indices $1, \dots, \ell$
- **Theorem:** Σ^* is countable.

Countability

- **Theorem:** $\mathbb{N} \times \mathbb{N}$ is countable.



- **Theorem:** The countable union of countable sets is countable.
 - List each countable set as a row in an infinite grid
 - Proof by picture (list elements along the diagonal)

Uncountability: Proof by Contradiction

- **Theorem:** The power set $\mathcal{P}(\mathbb{N})$ of natural numbers is **not** countable.
- **Proof Outline:**
 - Assume that $\mathcal{P}(\mathbb{N})$ is countable: a bijection to $\mathbb{N}$ exists
 - That is, can enumerate all sets in $\mathcal{P}(\mathbb{N})$ as $S_1, S_2, \dots, S_k, \dots$
 - Reach an absurdity/contradiction $\Rightarrow \Leftarrow$

Uncountability: Proof by Contradiction

- **Theorem:** The power set $\mathcal{P}(\mathbb{N})$ of natural numbers is **not** countable.
- **Proof:**
- Consider the set $D = \{i \in \mathbb{N} \mid i \in S_i\}$
 - Corresponds to the diagonal **Y** elements

$S_i \backslash j$	1	2	3	4	5	...
S_1	Y	N	N	Y	N	...
S_2	N	N	Y	N	N	...
S_3	Y	N	Y	N	Y	...
S_4	N	N	N	N	N	...
S_5	N	N	Y	Y	Y	...
...	...	...	...	...	...	...
S_k	N	Y	N	N	N	...

Uncountability: Proof by Contradiction

- **Theorem:** The power set $\mathcal{P}(\mathbb{N})$ of natural numbers is **not** countable.
- **Proof:**
- Consider the set $\overline{D} = \{i \in \mathbb{N} \mid i \notin S_i\}$
 - Corresponds to the diagonal **N** elements

$S_i \backslash j$	1	2	3	4	5	...
S_1	Y	N	N	Y	N	...
S_2	N	N	Y	N	N	...
S_3	Y	N	Y	N	Y	...
S_4	N	N	N	N	N	...
S_5	N	N	Y	Y	Y	...
...	...	...	...	...	...	...
S_k	N	Y	N	N	N	...

Uncountability: Proof by Contradiction

- **Theorem:** The power set $\mathcal{P}(\mathbb{N})$ of natural numbers is **not** countable.
- **Proof:**
 - $\overline{D} = \mathbb{N} - D$, that is, $\overline{D}$ is a valid subset of $\mathbb{N}$: $\exists k$ such that $S_k = \overline{D}$
 - What is the contradiction?

$S_i \backslash j$	1	2	3	4	5	...
S_1	Y	N	N	Y	N	...
S_2	N	N	Y	N	N	...
S_3	Y	N	Y	N	Y	...
S_4	N	N	N	N	N	...
S_5	N	N	Y	Y	Y	...
...	...	...	...	...	...	...
S_k	N	Y	N	N	N	...

Uncountability: Proof by Contradiction

- **Theorem:** The power set $\mathcal{P}(\mathbb{N})$ of natural numbers is **not** countable.

- **Proof:**

- If $k \in S_k \implies k \in D \implies k \notin \bar{D} = S_k$
- If $k \notin S_k \implies k \notin D \implies k \in \bar{D} = S_k$

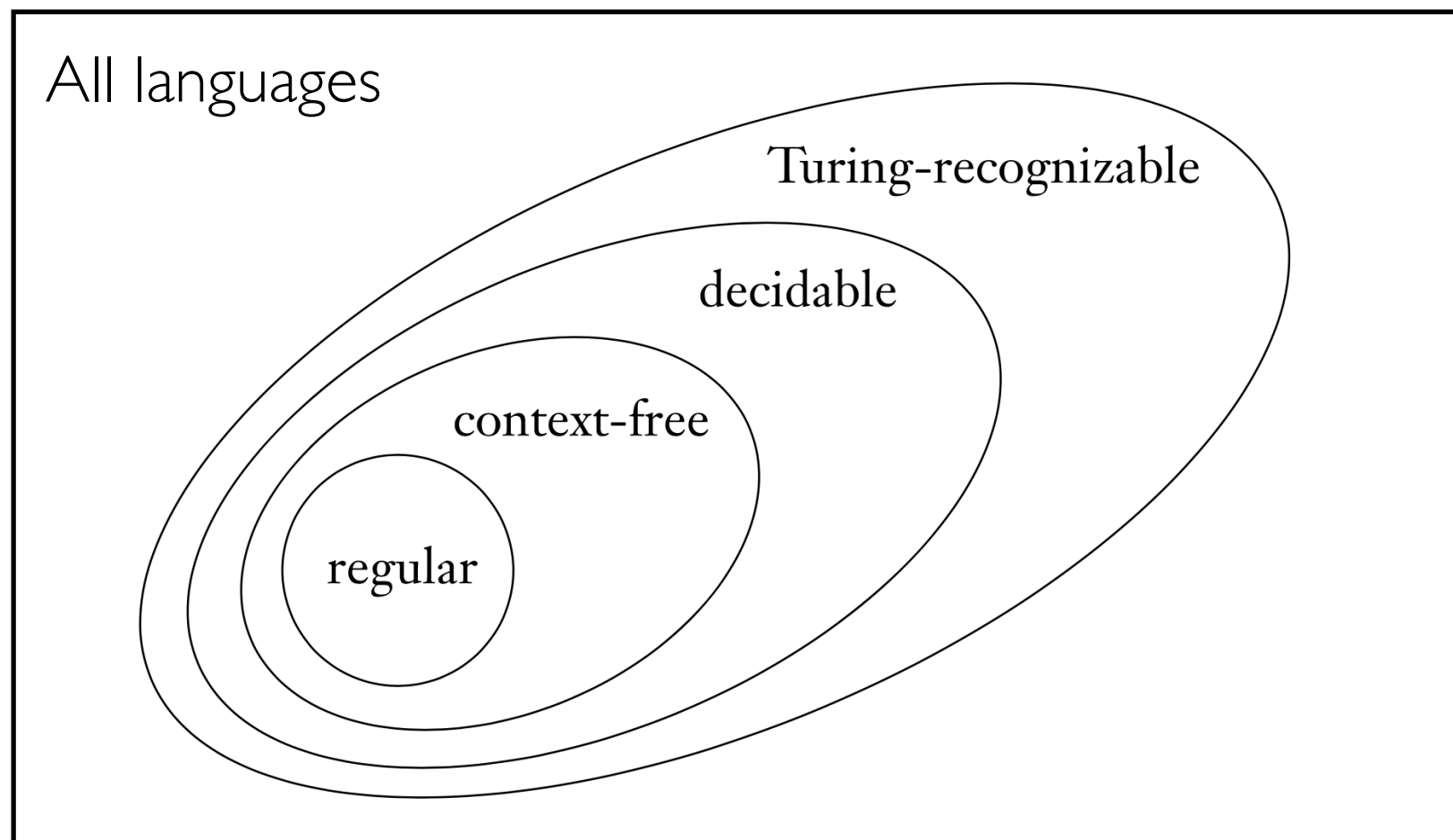
$S_i \backslash j$	1	2	3	4	5	...
S_1	Y	N	N	Y	N	...
S_2	N	N	Y	N	N	...
S_3	Y	N	Y	N	Y	...
S_4	N	N	N	N	N	...
S_5	N	N	Y	Y	Y	...
...	...	...	...	...	...	...
S_k	N	Y	N	N	N	...

Cantor's Diagonalization Argument

- This is an example of diagonalization argument
- Visualize where S_k appears on the diagonal
- ***Fun soviet version:***
 - <https://algorithmsoup.wordpress.com/2018/09/18/soviet-version-of-cantors-diagonalization-argument/>

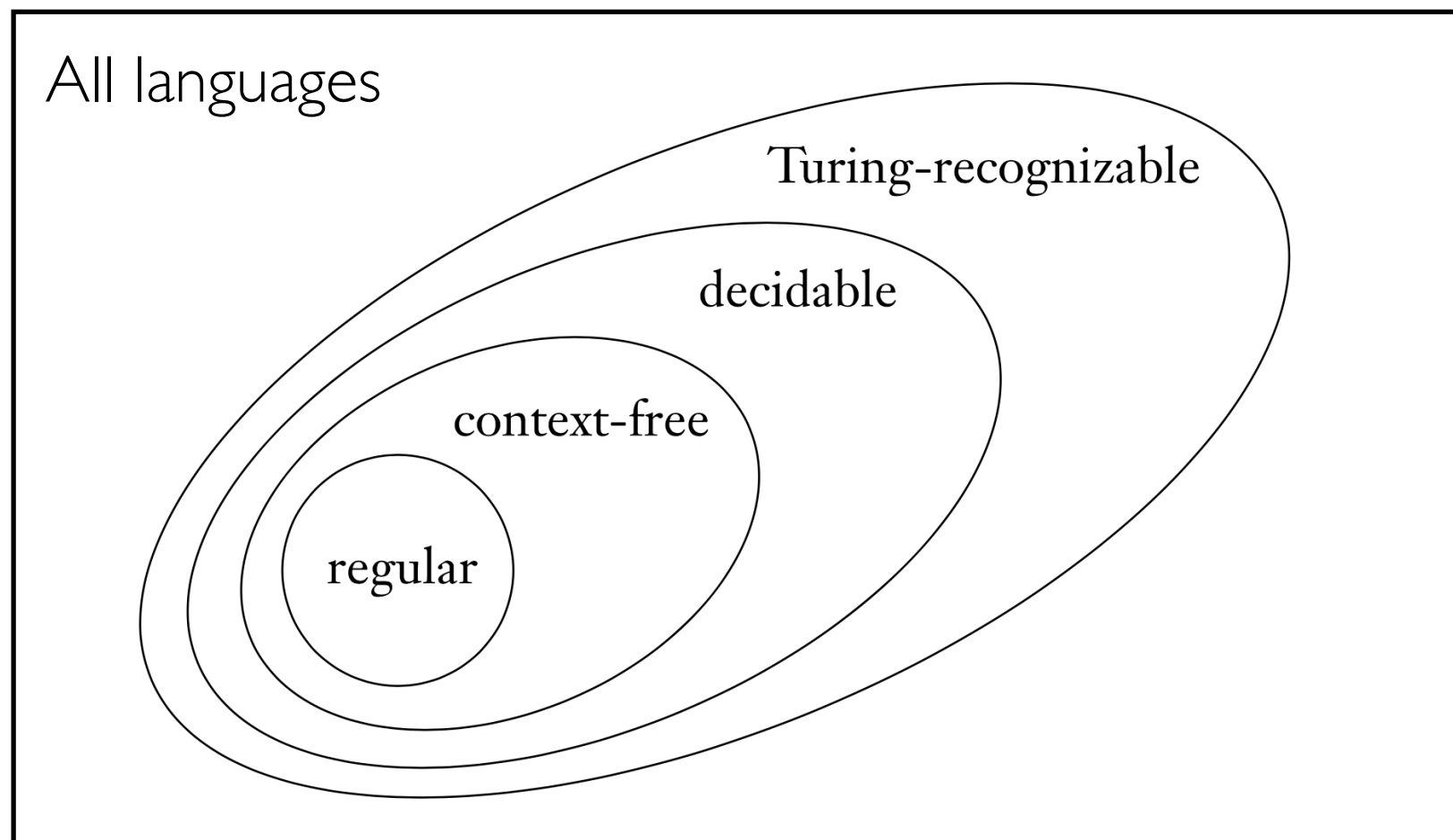
Countability Argument

- There are **many** languages that cannot even be recognized by TMs
- Can argue by comparing set of all TMs to set of all languages



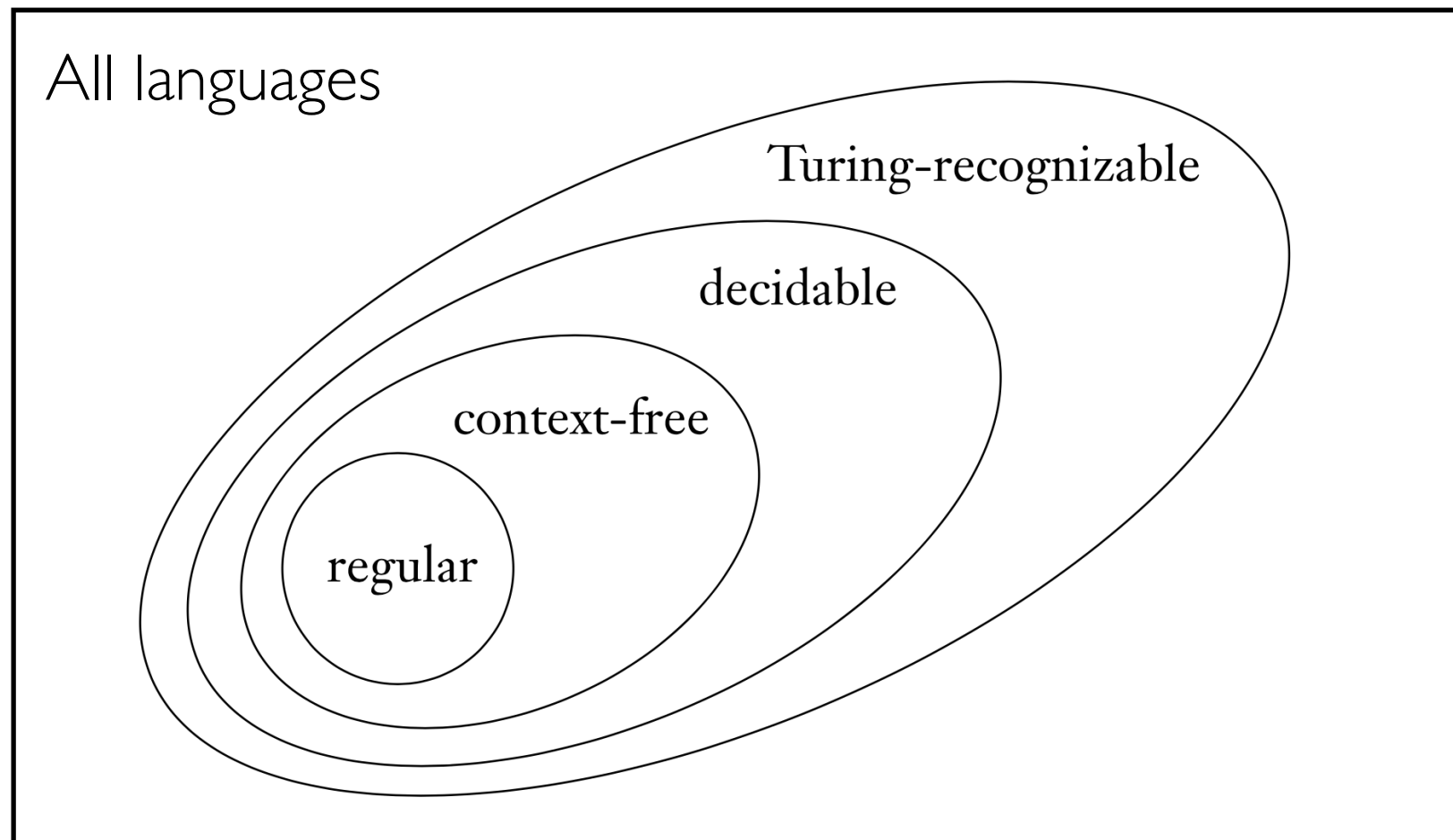
Countability Argument

- **Question.** Why is the set of all TM's countable?
- Σ^* is countable for any finite alphabet
- TM's can be encoded: $M \rightarrow \langle M \rangle$ over a finite Σ



Countability Argument

- **Question.** Why is the **set of all languages** $\mathcal{L}$ uncountable?
 - Mapping between $\mathcal{L}$ and set of infinite binary sequences
 - Alternatively: $\mathcal{L}$ is the power set of Σ^*



Uncountably Many Languages

- A language $L \subseteq \Sigma^*$
- The set of all languages $\mathcal{L}$ over Σ is the power set of Σ^*
 - Can prove is uncountable similar to power set of $\mathbb{N}$
- **Claim.** (Prove in HW 7) The set of all **infinite binary sequences** $\mathcal{B}$ (unending sequence of 0s and 1s) is uncountable by diagonalization.
- Can find a bijective map from the set of all languages over a finite alphabet Σ is $\mathcal{B}$
 - Set $\Sigma^* = \{s_1, s_2, \dots, s_k, \dots\}$
 - Map $f: \mathcal{L} \rightarrow \mathcal{B}$ where $f(L)$ is an infinite binary sequence where the i th bit is 1 if $s_i \in L$ and 0 otherwise
 - f is a bijection

Countability Argument

- **Takeaway:** There are infinitely many decision problems that cannot be solved by any TM
- **Next time:** Specific problems that are undecidable and unrecognizable

